

Protótipo de Digital Twin com Modelo Longitudinal Simplificado

Estimativa Energética em Ambiente CARLA

Camila Félix dos Reis

Março 2026

Sumário

1	Introdução	2
2	Levantamento de Requisitos	3
2.1	Requisitos Funcionais	3
2.2	Requisitos Não Funcionais	4
2.3	Estado Atual do Projeto	5
2.4	Evolução Implementada	6
2.5	Estrutura de Submodelos AAS	6
2.6	Aquisição de Variáveis do CARLA	8
3	Fundamentação Teórica	9
3.1	Digital Twin	9
3.2	Modelo Longitudinal Simplificado	9
3.3	Convenção de Sinal e Regime Regenerativo	10
3.4	Parâmetros do Modelo	11
4	Metodologia	12
5	Arquitetura do Sistema	13
5.1	Visão Geral Arquitetural	13
5.2	Diagrama de Componentes	13
5.3	Responsabilidades por Camada	14
5.3.1	Camada de Simulação	14
5.3.2	Camada de Domínio	14
5.3.3	Camada de Aplicação	14
5.3.4	Camada de Visualização	15
5.4	Fluxo de Execução	15
5.5	Interfaces entre Camadas	15
5.5.1	Interface Simulação → Modelo	15
5.5.2	Interface Modelo → Visualização	15
5.6	Justificativa Arquitetural	16
6	Resultados Esperados	17
7	Conclusão	18

Capítulo 1

Introdução

Simuladores veiculares como o CARLA fornecem variáveis dinâmicas do veículo, porém não integram explicitamente um modelo energético para estimativa de consumo em tempo real.

Este trabalho propõe o desenvolvimento de um protótipo de Digital Twin energético baseado em um modelo longitudinal simplificado para estimativa de potência instantânea e energia acumulada.

Capítulo 2

Levantamento de Requisitos

Este capítulo apresenta os requisitos funcionais e não funcionais do protótipo de Digital Twin energético.

O sistema possui implementação completa do modelo longitudinal, modelo simplificado de bateria, coordenação energética com indicadores estratégicos, integração com o simulador CARLA e sincronização com Asset Administration Shell (AAS) via Eclipse BaSyx. A integração via OPC UA está prevista para etapa subsequente.

2.1 Requisitos Funcionais

RF1 – Cálculo de Forças Longitudinais

O sistema deve calcular a força longitudinal equivalente considerando componente inercial, resistência ao rolamento e arrasto aerodinâmico.

RF2 – Cálculo de Potência Mecânica

O sistema deve calcular a potência mecânica instantânea a partir da força longitudinal e da velocidade do veículo.

RF3 – Integração Discreta de Energia

O sistema deve integrar numericamente a energia mecânica acumulada utilizando aproximação discreta baseada no passo de tempo Δt .

RF4 – Atualização do Estado de Carga (SoC)

O sistema deve atualizar o estado de carga da bateria com base na energia elétrica consumida ou regenerada, considerando eficiências simplificadas de descarga e regeneração.

RF5 – Coordenação Energética Unificada

O sistema deve disponibilizar um método unificado `update(request)` por meio da classe `VehicleEnergySystem`, retornando:

- Potência instantânea

- Energia mecânica acumulada
- Estado de carga (SoC)
- Energia elétrica utilizada ou regenerada
- Distância acumulada
- Potência média móvel
- Consumo específico
- Autonomia estimada

RF6 – Integração com Simulador

O sistema deve ser capaz de receber como entrada:

- Velocidade longitudinal
- Aceleração longitudinal
- Intervalo de tempo

A integração direta com a API do CARLA é realizada na camada de infraestrutura, mantendo o domínio desacoplado.

RF7 – Integração com Representação Digital Twin (AAS)

O sistema deve permitir integração com uma instância de Asset Administration Shell (AAS), implementada via Eclipse BaSyx, representando formalmente o Digital Twin energético.

RF8 – Comunicação via OPC UA

O sistema deve permitir exposição ou consumo de variáveis energéticas por meio do protocolo OPC UA, possibilitando integração com infraestrutura industrial ou middleware externo.

A mediação entre o modelo energético e a infraestrutura AAS/OPC UA será realizada pela camada de aplicação, por meio da classe `TwinService`.

2.2 Requisitos Não Funcionais

RNF1 – Independência do Domínio

A camada de domínio não deve possuir dependência direta do simulador CARLA, BaSyx ou bibliotecas externas de infraestrutura.

RNF2 – Testabilidade Isolada

O modelo energético deve ser executável de forma independente, permitindo validação offline sem necessidade de simulação ou middleware externo.

RNF3 – Separação de Responsabilidades

A arquitetura deve manter separação clara entre:

- Simulação
- Aplicação
- Domínio
- Infraestrutura
- Visualização

RNF4 – Execução Compatível com Passo Discreto

O cálculo energético deve ser executado em tempo compatível com o passo discreto da simulação, garantindo sincronização consistente.

RNF5 – Extensibilidade Arquitetural

O sistema deve permitir evolução futura para:

- Inclusão de inclinação da via
- Modelos avançados de bateria
- Estratégias energéticas mais complexas
- Integração industrial via OPC UA
- Representação formal completa em AAS

2.3 Estado Atual do Projeto

O projeto encontra-se na seguinte fase:

- Modelo longitudinal implementado e validado
- Modelo simplificado de bateria implementado e validado
- Sistema coordenador energético com indicadores estratégicos implementado
- Integração com CARLA implementada
- Integração com AAS via Eclipse BaSyx implementada
- Sistema de logging estruturado implementado
- 29 testes unitários cobrindo a camada de domínio
- Simulação sintética offline funcional
- Estrutura arquitetural definida segundo Clean Architecture
- Camada de visualização (HUD) prevista para próxima etapa
- Integração OPC UA prevista para próxima etapa

2.4 Evolução Implementada

Desde a versão inicial do documento, as seguintes evoluções foram implementadas no sistema:

Sistema de Logging Estruturado

Foi implementado um sistema centralizado de logging utilizando a biblioteca `structlog`, configurado na camada de infraestrutura (`src/infra/logger.py`). A camada de domínio permanece livre de dependências de logging, conforme exigido pelo requisito RNF1. O logging estruturado permite auditoria completa do ciclo de execução, registrando variáveis energéticas a cada passo de simulação.

Integração com Asset Administration Shell (BaSyx)

A classe `TwinService` foi implementada na camada de aplicação, realizando sincronização em tempo real com uma instância de Asset Administration Shell (AAS) via Eclipse BaSyx v2. A comunicação utiliza a API REST do BaSyx, atualizando os submodelos:

- **OperationalState**: velocidade, aceleração, potência instantânea e energia acumulada
- **Battery**: estado de carga (SoC) e fluxo de energia elétrica

O sistema verifica a disponibilidade do servidor BaSyx durante a inicialização e opera de forma degradada caso o serviço não esteja acessível, mantendo o ciclo de simulação ativo.

2.5 Estrutura de Submodelos AAS

A Asset Administration Shell do veículo energético é composta por três submodelos, cujos identificadores e mapeamentos são apresentados na Tabela 2.1.

Tabela 2.1: Mapeamento de variáveis do modelo para submodelos AAS

Submodelo	Elemento (idShort)	Variável do Modelo	Unidade
OperationalState	Velocity	v_{long}	m/s
OperationalState	Acceleration	a_{long}	m/s ²
OperationalState	InstantaneousPower	P	W
OperationalState	AccumulatedEnergy	E	J
Battery	StateOfCharge	SoC	%
Battery	ElectricalEnergyFlow	E_{elec}	J
Battery	NominalCapacity	C_{bat}	J
TechnicalData	Mass	m	kg
TechnicalData	DragCoefficient	C_d	–
TechnicalData	FrontalArea	A	m ²
TechnicalData	RollingResistance	C_r	–

A comunicação é realizada via API REST do Eclipse BaSyx v2, utilizando o endpoint:

PATCH /submodels/{base64(submodelId)}/submodel-elements/{idShort}/\$value

Os identificadores dos submodelos são codificados em Base64 URL-safe conforme especificação AAS Part 2.

Indicadores Estratégicos de Eficiência

A classe `VehicleEnergySystem` foi estendida com o cálculo de indicadores estratégicos de eficiência energética:

- **Distância acumulada:** integração discreta da velocidade ao longo do tempo

$$d(t) = \sum_{k=0}^n |v_k| \cdot \Delta t \quad (2.1)$$

- **Potência média móvel:** média das últimas N amostras (janela de 50 passos, correspondente a 5 segundos)

$$\bar{P}(t) = \frac{1}{N} \sum_{i=t-N}^t P(i) \quad (2.2)$$

- **Consumo específico:** relação entre energia elétrica consumida e distância percorrida

$$C_{esp} = \frac{E_{elec}/3600}{d} \quad [\text{Wh/m}] \quad (2.3)$$

- **Autonomia estimada:** tempo restante estimado com base na energia disponível e potência média

$$A(t) = \frac{SoC \cdot C_{bat}}{\bar{P}(t)} \quad [\text{s}] \quad (2.4)$$

Validação por Testes Unitários

Foram implementados 29 testes unitários cobrindo as três classes do domínio, validando:

- **EnergyModel:** cálculo de forças, potência, acumulação de energia e validação de parâmetros
- **Battery:** descarga, regeneração, limitação do SoC e validação de parâmetros
- **VehicleEnergySystem:** indicadores estratégicos, acumulação de distância, convergência da média móvel e comportamento com veículo estacionário

A validação offline do modelo confirma o atendimento ao requisito RNF2 (testabilidade isolada).

Simulação Sintética

Foi desenvolvido um script de simulação sintética (`scripts/simulate.py`) que executa o pipeline energético completo sem dependência do CARLA ou BaSyx, utilizando um perfil de velocidade com três fases: aceleração (0–15s), cruzeiro (15–40s) e frenagem regenerativa (40–60s).

Os resultados confirmam comportamento coerente: redução progressiva do SoC durante aceleração e cruzeiro, recuperação parcial durante frenagem regenerativa, e convergência dos indicadores estratégicos.

2.6 Aquisição de Variáveis do CARLA

O simulador CARLA fornece velocidade e aceleração como vetores tridimensionais no referencial do mundo. Para obtenção das componentes longitudinais, é realizada a projeção no vetor unitário de direção do veículo (*forward vector*):

$$v_{long} = \vec{v} \cdot \hat{f} = v_x f_x + v_y f_y + v_z f_z \quad (2.5)$$

$$a_{long} = \vec{a} \cdot \hat{f} = a_x f_x + a_y f_y + a_z f_z \quad (2.6)$$

Onde $\hat{f}$ é o vetor unitário de orientação frontal do veículo, obtido via `vehicle.get_transform().get`

Esta projeção garante que apenas a componente longitudinal seja considerada no modelo energético, descartando componentes laterais e verticais inerentes à dinâmica do simulador. Não foi aplicado filtro temporal sobre a aceleração; em cenários com ruído significativo, recomenda-se a aplicação de um filtro passa-baixa (e.g., média móvel ou Butterworth de 1^a ordem).

Capítulo 3

Fundamentação Teórica

3.1 Digital Twin

O conceito de *Digital Twin* foi inicialmente formalizado por Grieves [3], sendo posteriormente expandido como uma integração dinâmica entre sistemas físicos e modelos computacionais conectados em tempo real [2].

Um Digital Twin é uma representação virtual dinâmica de um sistema físico, capaz de refletir seu comportamento de forma sincronizada com a simulação.

Neste trabalho considera-se apenas dinâmica longitudinal em plano horizontal, desconsiderando inclinação da via.

3.2 Modelo Longitudinal Simplificado

Modelos longitudinais simplificados para análise de forças e potência são amplamente discutidos na literatura de dinâmica veicular [5]. O modelo considera as principais forças atuantes no veículo:

$$F_{rol} = C_r m g \quad (3.1)$$

$$F_{aero} = \frac{1}{2} \rho C_d A v^2 \quad (3.2)$$

O modelo considera a força longitudinal equivalente necessária para sustentar o estado dinâmico observado:

$$F_{long} = m \cdot a + F_{rol} + F_{aero} \quad (3.3)$$

Onde:

- m é a massa do veículo [kg]
- a é a aceleração longitudinal [m/s²]
- C_r é o coeficiente de resistência ao rolamento
- g é a aceleração da gravidade [m/s²]
- ρ é a densidade do ar [kg/m³]

- C_d é o coeficiente aerodinâmico
- A é a área frontal do veículo [m²]
- v é a velocidade longitudinal [m/s]

A modelagem simplificada de bateria adotada neste trabalho segue abordagem energética de alto nível, conforme discutido em sistemas de veículos elétricos [4].

A potência mecânica é dada por:

$$P = F_{long} \cdot v \quad (3.4)$$

A energia acumulada é obtida por integração temporal:

$$E = \int P dt \quad (3.5)$$

Na implementação computacional, a integração é realizada de forma discreta:

$$E_k = E_{k-1} + P_k \cdot \Delta t \quad (3.6)$$

3.3 Convenção de Sinal e Regime Regenerativo

A convenção de sinal adotada no modelo é:

- $P > 0$: regime de tração — energia é retirada da bateria
- $P = 0$: veículo estacionário ou em equilíbrio dinâmico
- $P < 0$: regime de frenagem regenerativa — energia é devolvida à bateria

No regime de tração, a energia elétrica consumida é calculada considerando a eficiência de descarga:

$$E_{elec} = \frac{P \cdot \Delta t}{\eta_d} \quad (3.7)$$

No regime regenerativo, a energia recuperada é atenuada pela eficiência de regeneração:

$$E_{regen} = |P| \cdot \Delta t \cdot \eta_r \quad (3.8)$$

A atualização do estado de carga segue:

$$SoC_{k+1} = SoC_k - \frac{E_{elec}}{C_{bat}} \quad (\text{tração}) \quad (3.9)$$

$$SoC_{k+1} = \min \left(1, SoC_k + \frac{E_{regen}}{C_{bat}} \right) \quad (\text{regeneração}) \quad (3.10)$$

O estado de carga é limitado ao intervalo $[0, 1]$. Na implementação atual não há limite de potência regenerativa; toda energia mecânica negativa é convertida com eficiência η_r . Esta simplificação é adequada para o escopo do protótipo, podendo ser refinada com curvas de limite de corrente em versões futuras.

3.4 Parâmetros do Modelo

A Tabela 3.1 apresenta os parâmetros utilizados na configuração do modelo longitudinal e do modelo de bateria.

Tabela 3.1: Parâmetros do modelo energético

Parâmetro	Símbolo	Valor	Unidade
Massa do veículo	m	1500	kg
Coefficiente aerodinâmico	C_d	0,3	–
Área frontal	A	2,2	m ²
Coefficiente de rolamento	C_r	0,015	–
Densidade do ar	ρ	1,225	kg/m ³
Aceleração da gravidade	g	9,81	m/s ²
Capacidade da bateria	C_{bat}	50	MJ
Eficiência de descarga	η_d	0,9	–
Eficiência de regeneração	η_r	0,9	–
Passo de simulação	Δt	0,1	s

Os valores de m , C_d e A correspondem a um veículo de passageiros típico (sedan médio). Os coeficientes de eficiência da bateria representam valores conservadores para sistemas de armazenamento baseados em íons de lítio.

Capítulo 4

Metodologia

A implementação foi realizada em Python utilizando a API oficial do simulador CARLA, um ambiente de simulação urbana amplamente utilizado em pesquisas de veículos autônomos [1].

O sistema é estruturado em quatro camadas principais:

- Camada de Simulação (aquisição de dados do CARLA)
- Camada de Aplicação (orquestração e sincronização com AAS)
- Camada de Domínio (modelo energético longitudinal)
- Camada de Visualização (HUD e/ou logger)

Capítulo 5

Arquitetura do Sistema

5.1 Visão Geral Arquitetural

A arquitetura do Digital Twin energético foi estruturada segundo o princípio de separação de responsabilidades (Separation of Concerns), organizando o sistema em camadas independentes e desacopladas.

O objetivo principal da arquitetura é garantir desacoplamento entre o modelo energético e o simulador CARLA, permitindo testabilidade isolada da camada de domínio e futura substituição da fonte de dados sem impacto na lógica do modelo.

5.2 Diagrama de Componentes

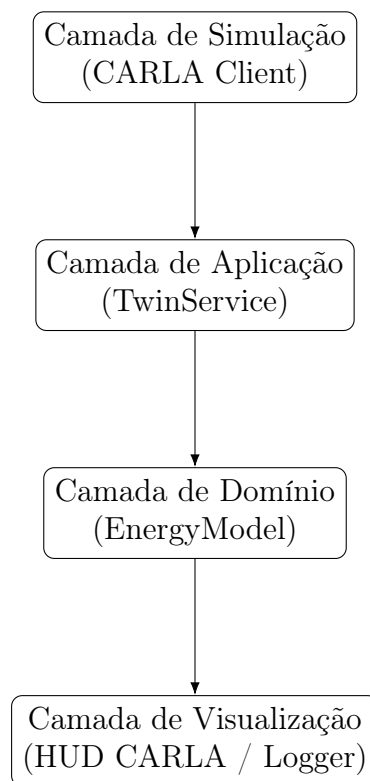


Figura 5.1: Arquitetura simplificada do Digital Twin energético

A sincronização temporal discreta e consistente entre o ambiente de simulação e o modelo energético caracteriza o sistema como um protótipo de Digital Twin baseado em simulação, uma vez que existe um ativo (simulado), um modelo digital correspondente e atualização consistente ao longo do tempo.

5.3 Responsabilidades por Camada

5.3.1 Camada de Simulação

Responsável por:

- Obter velocidade (v)
- Obter aceleração (a)
- Obter passo de tempo (dt)

Esta camada não realiza cálculos energéticos.

5.3.2 Camada de Domínio

Implementa o modelo longitudinal simplificado e é responsável por:

- Cálculo das forças longitudinais
- Cálculo da potência instantânea
- Integração discreta da energia acumulada
- Atualização da energia mecânica acumulada
- Interação com o modelo de bateria para atualização do SoC

Importante: esta camada não possui dependência direta do CARLA.

5.3.3 Camada de Aplicação

Responsável por orquestrar o ciclo de execução:

1. Receber dados da simulação
2. Invocar o método `update(v,a,dt)`
3. Encaminhar resultados para visualização

5.3.4 Camada de Visualização

Responsável por apresentar as variáveis energéticas calculadas pelo modelo, podendo ser implementada como:

- HUD (Head-Up Display) integrado ao ambiente do CARLA
- Saída em console
- Registro em arquivo (CSV/JSON)

Esta camada apenas consome dados produzidos pelo modelo energético, não realizando cálculos ou modificações no estado interno.

5.4 Fluxo de Execução

O fluxo operacional simplificado é:

Aquisição de Dados → Processamento do Modelo → Atualização de Estado Interno → Visualização

5.5 Interfaces entre Camadas

5.5.1 Interface Simulação → Modelo

A comunicação ocorre por meio das variáveis:

- Velocidade (v)
- Aceleração (a)
- Intervalo de tempo (dt)

O modelo expõe o método:

```
update(v: float, a: float, dt: float)
```

5.5.2 Interface Modelo → Visualização

O sistema energético retorna os resultados encapsulados em um objeto de resposta contendo:

- Potência instantânea
- Energia mecânica acumulada
- Estado de carga (SoC)
- Energia elétrica utilizada ou regenerada
- Distância acumulada
- Potência média móvel
- Consumo específico (Wh/m)
- Autonomia estimada (s)

5.6 Justificativa Arquitetural

A arquitetura adotada segue o princípio de separação de responsabilidades, isolando o modelo físico-matemático da infraestrutura de simulação. Essa abordagem permite validação independente do modelo energético, facilita a realização de testes unitários e reduz o acoplamento entre componentes.

Além disso, a independência do domínio possibilita futura substituição da fonte de dados (por exemplo, integração com hardware real ou middleware industrial) sem alterações na lógica do modelo.

Capítulo 6

Resultados Esperados

O protótipo implementado apresenta:

- Estimativa energética coerente com o comportamento dinâmico do veículo, validada por 29 testes unitários e simulação sintética offline
- Indicadores estratégicos de eficiência calculados em tempo real (consumo específico, autonomia estimada)
- Sincronização com Asset Administration Shell (AAS) via Eclipse BaSyx, caracterizando formalmente o Digital Twin energético
- Base estruturada para evolução futura, incluindo camada de visualização (HUD) e integração industrial via OPC UA

Capítulo 7

Conclusão

O protótipo desenvolvido integra modelagem física simplificada com ambiente de simulação realista, demonstrando viabilidade de estimativa energética em tempo real para veículos elétricos simulados. A arquitetura desacoplada e a representação formal via AAS estabelecem base para evolução futura em direção a sistemas embarcados e aplicações industriais.

Bibliografia

- [1] Alexey Dosovitskiy et al. “CARLA: An Open Urban Driving Simulator”. Em: *Conference on Robot Learning (CoRL)*. 2017, pp. 1–16.
- [2] Aidan Fuller et al. “Digital Twin: Enabling Technologies, Challenges and Open Research”. Em: *IEEE Access* 8 (2020), pp. 108952–108971. DOI: 10.1109/ACCESS.2020.2998358.
- [3] Michael Grieves. *Digital Twin: Manufacturing Excellence through Virtual Factory Replication*. White paper. 2014.
- [4] James Larminie e John Lowry. *Electric Vehicle Technology Explained*. 2^a ed. Wiley, 2012.
- [5] Rajesh Rajamani. *Vehicle Dynamics and Control*. 2^a ed. New York: Springer, 2011. DOI: 10.1007/978-1-4614-1433-9.